

Теоретический материал: Вложенные циклы

Вложенный цикл - это цикл, который находится внутри другого цикла. Вложенные циклы полезны, когда нужно обработать несколько уровней данных или создать таблицы, матрицы и рисунки.

Пример вложенного цикла:

```
for i in range(3):  
    for j in range(2):  
        print(i, j)
```

Этот код выводит все комбинации значений *i* и *j*. Для каждой итерации внешнего цикла *i* внутренний цикл *j* выполняется полностью.

Принцип работы вложенных циклов

Вложенные циклы работают следующим образом:

- Внешний цикл выполняет одну итерацию.
- Каждый раз, когда внешний цикл выполняется, внутренний цикл начинает свою полную работу.
- Внутренний цикл выполняется от начала до конца для каждой итерации внешнего цикла.

Пример:

```
for i in range(1, 4):  
    for j in range(1, 4):  
        print(f"i={i}, j={j}")
```

Здесь для каждого значения *i* от 1 до 3, внутренний цикл *j* также перебирает значения от 1 до 3.

Примеры использования вложенных циклов

Таблица умножения: Вложенные циклы можно использовать для построения таблиц умножения:

```
for i in range(1, 6):  
    for j in range(1, 6):  
        print(f"{i} * {j} = {i * j}", end="\t")  
    print() # Переход на новую строку
```

Рисование прямоугольника: Вложенные циклы можно использовать для рисования фигур. Например, для вывода прямоугольника из символов *:

```
width = 5
height = 3
for i in range(height):
    for j in range(width):
        print("*", end=" ")
    print() # Переход на новую строку после каждой строки
```

Примеры задач с вложенными циклами

Задача 1: Квадрат чисел

Напишите программу, которая выводит квадрат из чисел, где каждая строка содержит одинаковые числа.

```
for i in range(1, 4):
    for j in range(3):
        print(i, end=" ")
    print()
```

Результат:

```
1 1 1
2 2 2
3 3 3
```

Задача 2: Треугольник из звёздочек

Напишите программу, которая выводит треугольник из символов *:

```
for i in range(1, 6):
    for j in range(i):
        print("*", end=" ")
    print()
```

Результат:

```
*  
**  
***  
****  
*****
```

Задание 3: Напишите программу, которая выводит таблицу чисел от 1 до 10 в виде таблицы (каждый столбец — это число, умноженное на номера строк):

```
1  2  3  4  5  
2  4  6  8 10  
3  6  9 12 15  
4  8 12 16 20  
5 10 15 20 25
```

```
# 1. Внешний цикл: отвечает за СТРОКИ (множитель 1)  
# Перебирает числа от 1 до 5  
for row in range(1, 6):  
  
    # 2. Внутренний цикл: отвечает за СТОЛБЦЫ  
    (множитель 2)  
    # Перебирает числа от 1 до 5  
    for col in range(1, 6):  
  
        # 3. Вычисляем произведение  
        result = row * col  
  
        # 4. Выводим результат.  
        # Используем f-строку с форматированием.  
        # :<3 означает: выровнять число по левому краю и  
        выделить 3 символа  
        print(f"{result:<3}", end="")  
        # end="" гарантирует, что числа в одной строке
```

будут выведены без переноса

```
# 5. После завершения внутреннего цикла (т.е. после
вывода всех 5 столбцов),
# выводим пустую строку, чтобы перейти на новую
строку для следующего ряда.
print()
```

Инициализация игрового поля (Сетка/Карта) 🗺️

В геймдеве и при моделировании (например, в нейронных сетях или картографии) часто нужно создать сетку (матрицу) $M \times N$.

Сюжет: Создание шахматной доски 8x8.

Нужно пройти по каждой клетке, чтобы установить её начальное состояние (цвет, наличие фигуры).

- **Внешний цикл:** Перебирает **ряды** (от 0 до 7).
- **Внутренний цикл:** Перебирает **колонки** (от 0 до 7).

```
board_size = 8

print("Создание шахматной доски 8x8:")
for row in range(board_size):
    # Внешний цикл: строка
    row_output = ""
    for col in range(board_size):
        # Внутренний цикл: столбец

        # Установка цвета клетки: если сумма координат
        # четная - белая (W), нечетная - черная (B)
        if (row + col) % 2 == 0:
            row_output += " W "
        else:
            row_output += " B "

    print(row_output)
```

Цель: Пройти по всем **координатам** (x, y) или (row, col) на плоскости и выполнить действие для каждой точки.

Поиск дубликатов и сопоставление данных 🤝

В базах данных и аналитике часто нужно сравнить **каждый** элемент из одного списка с **каждым** элементом из другого списка (или того же самого).

Сюжет: Поиск всех возможных конфликтов в расписании (проверка пар).

У вас есть список студентов, и вы хотите убедиться, что ни у какой пары студентов нет одинаковых фамилий (в маленькой группе).

- **Внешний цикл:** Выбирает первого студента (**Student A**).
- **Внутренний цикл:** Сравнивает **Student A** со всеми остальными студентами (**Student B**).

```
students = ["Иванов", "Петров", "Сидоров", "Иванов",  
            "Кузнецов"]  
count = 0  
  
# Внешний цикл: выбирает первого студента  
for i in range(len(students)):  
  
    # Внутренний цикл: сравнивает его со всеми  
    # последующими студентами (j > i)  
    # Это предотвращает сравнение студента с самим собой  
    # и повторные проверки  
    for j in range(i + 1, len(students)):  
  
        student_a = students[i]  
        student_b = students[j]  
  
        if student_a == student_b:  
            print(f"Конфликт найден: Студенты {i+1} и  
{j+1} имеют одинаковую фамилию '{student_a}'")  
            count += 1  
  
print(f"Общее количество найденных конфликтов: {count}")
```

Цель: Выполнить операцию сопоставления $N \times N$, что характерно для алгоритмов сортировки, поиска дубликатов или сравнения всех возможных пар объектов.

Использование вложенных циклов в реальной жизни

- Матричные структуры: Вложенные циклы часто используются для работы с двумерными структурами данных, такими как таблицы и матрицы.
- Рисование фигур: Вложенные циклы полезны для создания различных графических фигур и структур (прямоугольники, треугольники, пирамиды).

Основные моменты:

1. Вложенные циклы позволяют организовать выполнение цикла внутри другого цикла. Это полезно при работе с несколькими уровнями данных или задачами, требующими многократной обработки.
2. Принцип работы:
 - Внешний цикл выполняет одну итерацию, после чего выполняется полный цикл внутреннего цикла.
 - Внутренний цикл завершает свою работу для каждой итерации внешнего цикла.
3. Примеры задач с вложенными циклами включают создание таблиц умножения, рисование фигур, работу с матрицами и другими многомерными структурами.